

subroutines or procedures and functions. In these languages, the formal arguments may be passed and returned only by the subroutines or procedures. The functions may take some formal arguments and can only return one value and then only through the function name itself.

In C++, there is no difference between a function and subroutine. The functions may or may not take some formal arguments for calling a portion of the program. The function may or may not transfer back, values to a called function block. It may behave like a traditional subroutine or as a function or as both. Secondly, most high level languages differentiate between a main program and subprograms. In C++, all modules are called functions and they all have the same structure.

4.2 DEFINING A FUNCTION

A function definition has a name, a parentheses pair containing zero or more parameters and a body. For each parameter, there should be a corresponding declaration that occurs before the body. Any parameter not declared is taken to be an int by default. It is good programming practice to declare all parameters.

Recent compilers such as ANSI C may permit including the argument declaration within the parentheses :

The general format of the function definition is given below.

```
functiontype functionname (datatype argument1,datatype argument2...)
{
    body of function
    _____
    _____
    return something
}
```

(a) **Declaring the type of a function** The function refers to the type of value it would return to the calling portion of the program. Any of the basic data types such as int, float, char etc. may appear in the function declaration. In many compilers, when a function is not supposed to return any value, it may be declared as type void, which informs the compiler not to save any temporary space for a value to be sent back to the calling program.

For example,

```
void function_name (...)  
int function_name (...)  
float function_name (...)  
char function_name (...)
```

(b) **Function name** The function name can be any name conforming to the syntax rules of the variables. Normally, a function name is made relevant to the function operation, as it will be easy to keep a track of it, whenever a transfer of similar functions is used in the main program.

For example,

```
counter ();  
square ();  
display_value();  
output ();
```

(c) **Formal arguments** Any variable declared in the body of a function is said to be local to that function. Other variables which are not declared either as arguments or in the function body, are considered "global" to the function and must be defined externally. The storage classes or scope of variables are discussed subsequently in this chapter.

For example,

(1)

```
void square (int a, int b)  
// a, b are the formal arguments  
{  
    _____  
    _____  
}
```

(2)

```
int counter ( float x1, float x2, int y1, int y2)  
{  
    // x1, x2, y1, and y2 are the formal arguments  
    _____  
    _____  
    return (int value);  
}
```

(3)

```
float output (void) // function without formal argument  
{
```

```

statement;
____
____
return(float value);
}

```

(d) **Function body** (After declaring the type of a function, function name and formal arguments, a statement or a block of statements is enclosed between the beginning and the end statements.) In C++, each function is declared almost like a `main()` function.

For example,

```

void all_add (int a, int b, int c, float x, float y)
{
    int i, j, n;    // local variables , if any
    _____
    _____
    _____
}

```

□ PROGRAM 4.1

A program to demonstrate a function declaration, function calling and function definition in C++.

```

#include <iostream.h>
void main (void)
{
    void display(void); // function declaration
    display(); // function calling
}
void display(void) // function definition
{
    cout << " this is a test program \n";
    cout << " for demonstrating a function call \n";
    cout << " in C++ \n";
}

```

Output of the above program

```

this is a test program
for demonstrating a function call
in C++

```

The main function invokes the `display()` function. In C++, each function is almost like a separate program. So the formal declaration such as type, begin and end must be used in the function also. The control will be transferred from the main function or from

a function to the called function. After it has executed all the statements in the function, the control switches back to the calling portion of the program.

4.3 **return** STATEMENT

The keyword `return` is used to terminate function and return a value to its caller. The `return` statement may also be used to exit a function without returning a value. The `return` statement may or may not include an expression.

The general syntax of the `return` statement is,

```
return;
return (expression );
```

The `return` is a full-fledged C++ statement that can appear anywhere within a function body. A function can also have more than one `return` although it is good programming style for a function to have a single entrance and a single exit.

Following are some valid `return` statements:

```
return;
return (54);
return (x+y);
return (++i);
return ++j; /* correct , but not good style */
```

The `return` statement terminates the execution of the function and pass on the control back to the calling environment.

For example, a few valid function declarations are,

```
(1)
int sum (int x,int y)
{
    return(x+y);    // return int value
}
```

```
(2)
float maximum(float a, float b)
{
    if ( a > b)
        return(a);
    else
        return(b);
} // return floating point value
```

□ PROGRAM 4.2

A program to find the maximum of any three numbers using multiple `return` statements in a function definition.

```

    cin >> x >> y >> z;
    max = maximum (x,y,z);
    cout << " maximum = " << max;
}
float maximum (float a,float b,float c)
{
    if (a > b) {
        if (a > c)
            return(a);
        else
            return(c);
    }
    else {
        if (b > c)
            return(b);
        else
            return(c);
    }
} // end of function

```

Output of the above program

enter three numbers

4 5 6

maximum = 6

4.4 TYPES OF FUNCTIONS

The user defined functions may be classified in the following three ways based on the formal arguments passed and the usage of the return statement, and based on that, there are three types of user defined functions, Type 1 to Type 3.)

- (a) A function is invoked without passing any formal argument from the calling portion of a program and also the function does not return back any value to the called function.

- (b) A function is invoked with formal arguments from the calling portion of a program but the function does not return back any value to the calling portion.
- (c) A function is invoked with formal arguments from the calling portion of a program which returns back a value to the calling environment.

Type 1 It is the simplest way of writing a user defined function in C++. There is no data communication between the calling portion of a program and a called function block. The function is invoked by a calling environment by not passing any formal argument and the function also does not return back any value to the caller.

For example,

```
#include <iostream.h>
void main (void)
{
    void message (void) // function declaration
    message (); // function calling
}
void message (void) // function definition
{
    _____
    _____ // body of the function
    _____
}
```

Type 2 The second type of user defined function passes some formal arguments to a function but the function does not return back any value to the caller. It is an one-way data communication between a calling portion of the program and the function block.

For example,

```
#include <iostream.h>
void main (void)
{
    void power (int ,int); // function declaration
    int x,y;
    _____
    _____
    power (x,y); // function calling
}
void power (int x,int y)
{
    _____
    _____
}
```

// body of the function
// no value will be transferred back to the caller
}

PROGRAM 4.3

A program to find the square of its number using a function declaration without using the return statement.

```
//example 4.3
// passing formal arguments and no return statement
#include <iostream.h>
void main (void)
{
    void square(int); // function declaration
    int max;
    cout << "enter a value for n ? \n";
    cin >> max;
    for ( int i =0;i <= max-1; ++i)
        square(i);
}
void square (int n) // function definition
{
    float value;
    value = n*n;
    cout << " i = " << n << " square = " << value << endl;
}
```

Output of the above program

enter a value for n ?

4

i = 0 square = 0

i = 1 square = 1

i = 2 square = 4

i = 3 square = 9

Type 3 The third type of user defined function passes some formal arguments to a function from a calling portion of the program and the computed value, if any, is transferred back to the caller. Data are communicated between both the calling portion of a program and the function block.

For example,

```
#include <iostream.h>
```

```

void main (void)
{
    int output (int,int,char); // function declaration
    int x,y,temp;
    char ch;

    _____

    temp = output (x,y,ch);
}
int output (int a,int b, char s)
{
    int value;

    _____

    // body of the function
    return(something);
}

```

□ PROGRAM 4.4

A program to find the square of its number using a function declaration with the return statement.

```

// using return statement
#include <iostream.h>
void main (void)
{
    float square(float); // function declaration
    float i,max,value;
    max = 1.5;
    i = -1.5;
    while ( i <= max) {
        value = square(i);
        cout << " i = " << i << " square = " << value << endl;
        i = i + 0.5;
    }
}
float square (float n) // function definition
{
    float value;
    value = n*n;
    return(value);
}

```

4.5 ACTUAL AND FORMAL ARGUMENTS

The arguments may be classified under two groups, actual and formal arguments.

(a) **Actual arguments** An actual argument is a variable or an expression contained in a function call that replaces the formal parameter which is a part of the function declaration.

Sometimes, a function may be called by a portion of a program with some parameters and these parameters are known as the actual arguments.

For example,

```
#include <iostream.h>
void main()
{
    int x,y;
    void output (int x,int y); // function declaration
    _____
    _____
    output (x,y); // x and y are the actual arguments.
}
```

(b) **Formal arguments** Formal arguments are the parameters present in a function definition which may also be called as dummy arguments or the parametric variables. When the function is invoked, the formal parameters are replaced by the actual parameters.

For example,

```
#include <iostream.h>
void main()
{
    int x,y;
    void output (int x,int y);
    _____
    _____
    output (x,y);
}
void output (int a,int b) // formal or dummy arguments
{
    // body of the function
}
```

Formal arguments may be declared by the same name or by different names in calling a portion of the program or in a called function but the data types should be the same in both blocks.

For example, the following function declaration is invalid.

```
#include <iostream.h>
void main()
{
    void funct (int x,int y, char s1, char s2);
    int x,y;
    char s1,s2;

    _____
    _____
    funct(x,y,s1,s2);
}
funct ( char c1,char c2,int a,int b) // data type mismatch
{
    // body of a function
}
```

4.6 LOCAL AND GLOBAL VARIABLES

The variables in general may be classified as local or global variables.

(a) **Local variables** Identifiers declared as label, const, type, variables and functions in a block are said to belong to a particular block or function and these identifiers are known as the local parameters or variables. Local variables are defined inside a function block or a compound statement.

For example,

```
void funct (int i,int j)
{
    int k,m; // local variables

    _____
    _____
    // body of the function
}
```

The integer variables k and m are defined within a function block of the funct(). All the variables to be used within a function block must be either defined at the beginning of the block itself or before using in a statement. Local variables are referred only the particular part of a block or a function. Same variable name may be given to different parts of a function or a block and each variable will be treated as a different entity.

For example,

```
function1(float a, float b)
{
    float x,y; // local variable
```

```
x = 10.101;  
y = 20.20;
```



```
_____  
_____  
    // body of the function 1  
}  
function2 (int i, int j)  
{  
    float x,y;  // local to the function 2  
    x = 1.111;  
    y = 2.2222;  
    _____  
    _____  
    // body of the function 2  
}
```

(b) **Global variables** Global variables are variables defined outside the main function block. These variables are referred by the same data type and by the same name through out the program in both the calling portion of a program and in the function block. Whenever some of the variables are treated as constants in both the main and the function block, it is advisable to use global variables.

For example,

```
int x,y = 4; // global declaration  
void main(void)  
{  
    void function1();  
    x = 10;  
    _____  
    _____  
    function1 ();  
}  
function1( )  
{  
    int sum;  
    sum = x+y;  
    _____  
    _____  
}
```

□ PROGRAM 4.9

A program to find the sum of the given two numbers using the global variable declaration.

```
// global variable declaration
#include <iostream.h>
int x;
int y = 5;
void main(void)
{
    x = 10;
    void sum (void);
    sum();
}
void sum (void)
{
    int sum;
    sum = x+y;
    cout << " x = " << x << endl;
    cout << " y = " << y << endl;
    cout << " sum = " << sum << endl;
}
```

Output of the above program

```
x = 10
y = 5
sum = 15
```

4.7 DEFAULT ARGUMENTS

One of the most useful facilities available in C++ is the facility to define default argument values for functions. In the function prototype declaration, the default values are given. Whenever a call is made to a function without specifying an argument, the program will automatically assign values to the parameters from the default function prototype declaration. Default arguments facilitate easy development and maintenance of programs.

For example, the following program segment illustrates the default argument declaration:

```
#include <iostream.h>
void sum (int x = 10, int y = 20);
// function prototype declaration
void main() // with default aragument list
```

```

{
    int a,b;
    sum (); // function calling
    _____
    _____
}
void sum(int a1,int a2) // function definition
{
    int temp;
    temp = a1+a2; // a1 = 10 and a2 = 20 by default arguments
}

```

❑ PROGRAM 4.10

A program to find the sum of the given numbers using default argument declaration.

```

//default argument declaration
#include <iostream.h>
void sum ( int a,int b,int c = 6, int d = 10);
void main()
{
    int a,b,c,d;
    cout << " enter any two numbers \n";
    cin >> a >> b;
    sum (a,b); // sum of default values
}
void sum (int a1,int a2,int a3,int a4)
{
    int temp;
    temp = a1 + a2 + a3 + a4;
    cout << " a = " << a1 << endl;
    cout << " b = " << a2 << endl;
    cout << " c = " << a3 << endl;
    cout << " d = " << a4 << endl;
    cout << " sum = " << temp;
}

```

Output of the above program

```

enter any two numbers
11 21
a = 11
b = 21

```

	DESCRIPTION
<code>++ *ptr</code>	<code>++ (*ptr)</code> increment the content of the location pointed to by ptr depending on the type of the content
<code>-- *ptr</code>	<code>-- (*ptr)</code> decrement content of the location pointed to by ptr depending on the type of the content
<code>*ptr--</code>	<code>*(ptr--)</code> retrieve the content of the location pointed to by ptr and then decrement ptr
<code>* --ptr</code>	<code>* (--ptr)</code> decrement ptr, then retrieve the content of the new location pointed to by ptr
<code>(*ptr) --</code>	retrieve content *ptr of the location pointed to by ptr, then decrement the content of that location; ptr is not changed

6.3 POINTERS AND FUNCTIONS

Pointers are very much used in a function declaration. Sometimes only with a pointer a complex function can be easily represented and accessed. The use of the pointers in a function definition may be classified into two groups; they are call by value and call by reference.

6.3.1 Call by Value

Whenever a portion of the program invokes a function with formal arguments, control will be transferred from the main to the calling function and the value of the actual argument is copied to the function. Within the function, the actual value copied from the calling portion of the program may be altered or changed. When the control is transferred back from the function to the calling portion of the program, the altered values are not transferred back. This type of passing formal arguments to a function is technically known as call by value.

For example, the following program segment illustrates the use of the call by value.

```
#include <iostream.h>
void main (void)
{
    void funct (int x , int y); // function declaration
    int x,y,z;
```

```

x = 10;
y = 20;

_____

_____

funct (x,y); // call by value

_____

_____

z = x+y;      // z = 10 +20 = 30
}
void funct (int a, int b)
{
    int sum;
    a = a*a; // new values will not be copied to the main
    b = b*b;
}

```

□ PROGRAM 6.13

A program to display the contents of a variable before and after the function is invoked using a call by value.

```

//example 6.13
#include <iostream.h>
void main(void)
{
    int i;
    int function (int);
    i = 20;
    cout << "value of i before function call =" << i<<endl;
    function(i);
    cout << endl;
    cout << " value of i after function call = " << i<<endl;
}
int function (int i)
{
    int x ;
    x = 5*i;
    cout << " Inside the function, value of i = ";
    cout << x << endl;
    return (x);
}

```

Output of the above program

value of i before function call = 20
Inside the function, value of i = 100
value of i after function call = 20

❑ PROGRAM 6.14

A program to exchange the contents of two variables using a call by value.

```
// example 6.14
// call by value
#include <iostream.h>
void main(void)
{
    int x,y;
    void swap (int, int);
    x = 100;
    y = 20;
    cout << " values before swap () " << endl;
    cout << " x = " << x << " and y = " << y << endl;
    swap (x,y); // call by value
    cout << " values after swap () " << endl;
    cout << " x = " << x << " and y = " << y << endl;
}
void swap( int x, int y) // values will not be swapped
{
    int temp;
    temp = x;
    x = y;
    y = temp;
}
```

Output of the above program

```
values before swap ()
x = 100 and y = 20
values after swap ()
x = 100 and y = 20
```

Since the above function is declared call by value, the desired result is not obtained. As a single variable is transferred to the function, it protects the value of this variable from alteration within the function. On the other hand, it prevents the altered value being transferred back from the function to the calling portion of the program.

6.3.2 Call by Reference

When a function is called by a portion of a program, the address of the actual arguments are copied onto the formal arguments, though they may be referred by different variable names. The content of the variables that are altered within the function block are returned to the calling portion of a program in the altered form itself, as the formal and the actual arguments are referencing the same memory location or address. This is technically known as call by reference or call by address or call by location.)

When an argument is passed by value, the data item is copied to the function. Thus, any alteration made to the data item within the function is not carried over into the calling routine. When an arguments passed by reference (i.e., when a pointer is passed to the function), the address of the data item is passed to the function. The contents of that address can be accessed freely, either within a function or within a calling routine. Moreover, any change that is made to the data item (i.e., the contents of the address) will be recognized in both the function and the calling portion of the program. Thus, the use of the pointer as a function arguments permits the corresponding data item to be altered globally from within the function.

For example, the following program segment illustrates the use of call by reference.

```
#include <iostream.h>
void main (void)
{
    void funct (int *x, int *y); // function declaration
    int x,y,z;
    x = 10;
    y = 20;

    _____

    _____

    funct (&x,&y); // call by reference
    z = x+y;      // z = 20 + 40 = 60

    _____

    _____
}

void funct (int *a, int *b)
{
    *a = *a + *a; // new values will be copied to the main
    *b = *b + *b;
}
```

□ PROGRAM 6.15

A program to exchange the contents of two variables using call by reference.

```
// call by reference
#include <iostream.h>
void main(void)
{
    int x,y;
    void swap (int *x, int *y);
    x = 100;
    y = 20;
    cout << " values before swap () " << endl;
    cout << " x = " << x << " and y = " << y << endl;
    swap (&x,&y); // call by reference
    cout << " values after swap () " << endl;
    cout << " x = " << x << " and y = " << y << endl;
}
void swap( int *x, int *y) // values will be swapped
{
    int temp;
    temp = *x;
    *x = *y;
    *y = temp;
}
```

Output of the above program

```
values before swap ()
x = 100 and y = 20
values after swap ()
x = 20 and y = 100
```

□ PROGRAM 6.16

A program to exchange the contents of two variables using call by reference (version 2)

```
// call by reference
// swap function version 2.cpp
#include <iostream.h>
void main(void)
{
```

```
    int x,y;
    void swap (int &x, int &y);
```